# Chapter 16 (Segmentation) — Homework (Simulation)

Zhiyuan Pan

All runs use `segmentation.py` in `ostep-homework/vm-segmentation/`. Every command below can be rerun with `code/chapter16/run.sh`. For Q4 and Q5 I used a small helper, `code/chapter16/seg_exp.py`, that runs the simulator with many random seeds and counts how many of the generated addresses were valid.

How the simulator translates a virtual address (VA) in an address space of size `asize`:

- The **top bit** of the VA picks the segment: `VA < asize/2` → segment 0, `VA ≥ asize/2` → segment 1.
- **Segment 0 grows upward.** Valid if `VA < limit0`; then `PA = base0 + VA`.
- **Segment 1 grows downward.** Its base register (`-B`) is the *top* of the segment in physical memory. The offset is negative: `offset = VA - asize`. Valid if `offset ≥ -limit1`; then `PA = base1 + offset`.

---

## Q1

> First let's use a tiny address space to translate some addresses. Here's a simple set of parameters with a few different random seeds; can you translate the addresses?
>
> - `segmentation.py -a 128 -p 512 -b 0 -l 20 -B 512 -L 20 -s 0`
> - `segmentation.py -a 128 -p 512 -b 0 -l 20 -B 512 -L 20 -s 1`
> - `segmentation.py -a 128 -p 512 -b 0 -l 20 -B 512 -L 20 -s 2`

Setup: a 128-byte address space (7-bit addresses, so the top bit has value 64) in 512 bytes of physical memory.

- Segment 0: base 0, limit 20 → valid VAs 0–19, `PA = VA`.
- Segment 1: top at 512, limit 20 → physical bytes 492–511. Valid offsets are −20 … −1, i.e. VAs 108–127, with `PA = 512 + (VA - 128)`.

**Seed 0**

| VA | segment | calculation | result |
|---|---|---|---|
| 108 (0x6c) | 1 (108 ≥ 64) | offset 108 − 128 = −20 ≥ −20 | **valid, PA 492** |
| 97 (0x61) | 1 | offset −31 < −20 | **violation (seg 1)** |
| 53 (0x35) | 0 (53 < 64) | 53 ≥ limit 20 | **violation (seg 0)** |
| 33 (0x21) | 0 | 33 ≥ 20 | **violation (seg 0)** |
| 65 (0x41) | 1 | offset −63 < −20 | **violation (seg 1)** |

**Seed 1**

| VA | segment | calculation | result |
|---|---|---|---|
| 17 (0x11) | 0 | 17 < 20 | **valid, PA 17** |
| 108 (0x6c) | 1 | offset −20 | **valid, PA 492** |
| 97 (0x61) | 1 | offset −31 | **violation (seg 1)** |
| 32 (0x20) | 0 | 32 ≥ 20 | **violation (seg 0)** |
| 63 (0x3f) | 0 (63 < 64, top bit 0) | 63 ≥ 20 | **violation (seg 0)** |

**Seed 2**

| VA | segment | calculation | result |
|---|---|---|---|
| 122 (0x7a) | 1 | offset −6 | **valid, PA 506** |
| 121 (0x79) | 1 | offset −7 | **valid, PA 505** |
| 7 (0x07) | 0 | 7 < 20 | **valid, PA 7** |
| 10 (0x0a) | 0 | 10 < 20 | **valid, PA 10** |
| 106 (0x6a) | 1 | offset −22 < −20 | **violation (seg 1)** |

All 15 translations matched the `-c` output.

## Q2

> Now, let's see if we understand this tiny address space we've constructed (using the parameters from the question above). What is the highest legal virtual address in segment 0? What about the lowest legal virtual address in segment 1? What are the lowest and highest illegal addresses in this entire address space? Finally, how would you run `segmentation.py` with the `-A` flag to test if you are right?

| question | answer | why |
|---|---|---|
| highest legal VA in segment 0 | **19** | limit 0 = 20, so VAs 0–19 |
| lowest legal VA in segment 1 | **108** | 128 – 20 = 108 (offset –20) |
| lowest illegal VA | **20** | first VA past segment 0 |
| highest illegal VA | **107** | last VA before segment 1 |

So the valid ranges are 0–19 and 108–127, and everything from 20 to 107 is illegal. VAs 20–63 fall in segment 0's half and 64–107 in segment 1's half.

To test it, pass exactly the boundary addresses with `-A`:

```
./segmentation.py -a 128 -p 512 -b 0 -l 20 -B 512 -L 20 -A 0,19,20,63,64,107,108,127 -c
```

```
VA  0: 0x00000000 (decimal:    0) --> VALID in SEG0: 0x00000000 (decimal:    0)
VA  1: 0x00000013 (decimal:   19) --> VALID in SEG0: 0x00000013 (decimal:   19)
VA  2: 0x00000014 (decimal:   20) --> SEGMENTATION VIOLATION (SEG0)
VA  3: 0x0000003f (decimal:   63) --> SEGMENTATION VIOLATION (SEG0)
VA  4: 0x00000040 (decimal:   64) --> SEGMENTATION VIOLATION (SEG1)
VA  5: 0x0000006b (decimal:  107) --> SEGMENTATION VIOLATION (SEG1)
VA  6: 0x0000006c (decimal:  108) --> VALID in SEG1: 0x000001ec (decimal:  492)
VA  7: 0x0000007f (decimal:  127) --> VALID in SEG1: 0x000001ff (decimal:  511)
```

The output confirms all four answers: 19 and 108 are the last/first legal addresses, and 20 and 107 are the first/last illegal ones.

## Q3

> Let's say we have a tiny 16-byte address space in a 128-byte physical memory. What base and bounds would you set up so as to get the simulator to generate the following translation results for the specified address stream: valid, valid, violation, ..., violation, valid, valid? Assume the following parameters:
>
> - `segmentation.py -a 16 -p 128 -A 0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15 --b0 ? --l0 ? --b1 ? --l1 ?`

VAs 0 and 1 must be valid (the start of segment 0), and VAs 14 and 15 must be valid (the end of segment 1, offsets –2 and –1). So **both limits are 2**. The bases only have to put the two segments somewhere in physical memory without overlapping, for example segment 0 at the bottom and segment 1 at the top:

```
./segmentation.py -a 16 -p 128 -A 0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15 --b0 0 --l0 2 -
-b1 128 --l1 2 -c
```

```
VA  0: ... --> VALID in SEG0: 0x00000000 (decimal:    0)
VA  1: ... --> VALID in SEG0: 0x00000001 (decimal:    1)
VA  2: ... --> SEGMENTATION VIOLATION (SEG0)
   ...       (VAs 3-7: violation in SEG0, VAs 8-13: violation in SEG1)
VA 13: ... --> SEGMENTATION VIOLATION (SEG1)
VA 14: ... --> VALID in SEG1: 0x0000007e (decimal:  126)
VA 15: ... --> VALID in SEG1: 0x0000007f (decimal:  127)
```

This gives exactly valid, valid, 12 violations, valid, valid. Any other bases work too, as long as segment 0 occupies `b0 … b0+1` and segment 1 occupies `b1-2 … b1-1` inside the 128 bytes without overlapping, because the base registers only change *where* the valid addresses land, not *which* addresses are valid.

## Q4

> Assume we want to generate a problem where roughly 90% of the randomly-generated virtual addresses are valid (not segmentation violations). How should you configure the simulator to do so? Which parameters are important to getting this outcome?

Random VAs are spread uniformly over the whole address space, and the valid ones are exactly the `limit0 + limit1` addresses covered by the two segments. So

$$\text{fraction valid} \approx \frac{\text{limit}_0 + \text{limit}_1}{\text{address-space size}}$$

To get 90%, choose the limits so that they cover 90% of the address space, e.g. with a 1 KB address space, `limit0 = limit1 = 461` (922 / 1024 = 90.0%). Each limit can be at most half the address space (`asize/2`), since each segment lives in one half.

Measured over 10 seeds × 1000 random addresses each:

| configuration | (l0 + l1) / asize | valid |
|---|---|---|
| `-a 1k -p 16k -b 0 -l 461 -B 16k -L 461` | 90.0% | **90.1%** |
| `-a 1k -p 16k -b 8k -l 461 -B 4k -L 461` (different bases) | 90.0% | **90.1%** |
| `-a 1k -p 16k -b 0 -l 512 -B 16k -L 410` (unequal limits) | 90.0% | **90.5%** |
| `-a 128 -p 512 -b 0 -l 58 -B 512 -L 57` (tiny address space) | 89.8% | **89.9%** |
| `-a 1k -p 16k -b 0 -l 256 -B 16k -L 256` (for comparison) | 50.0% | 50.3% |
| `-a 1k -p 16k` (random limits, default) | — | 78.4% |

**The important parameters are the two limits (`-l`, `-L`) relative to the address-space size (`-a`).** The bases (`-b`, `-B`) and the physical memory size (`-p`) do not matter at all: moving the segments elsewhere in physical memory gave exactly the same 9006 valid addresses. How the 90% is split between the two segments does not matter either. With the default random limits (each between asize/4 and asize/2), the valid fraction is somewhere between 50% and 100%, here 78.4%.

## Q5

> Can you run the simulator such that no virtual addresses are valid? How?

Yes: **set both limits to 0** (`-l 0 -L 0`). Then segment 0 has no valid VA (every `VA ≥ 0 = limit0`), and in segment 1 every offset `VA − asize` is at most −1, which is below −0. So every address is a violation, wherever the bases are.

```
./segmentation.py -a 128 -p 512 -b 0 -l 0 -B 512 -L 0 -n 10 -s 3 -c
```

```
VA  0: 0x0000001e (decimal:   30) --> SEGMENTATION VIOLATION (SEG0)
VA  1: 0x00000045 (decimal:   69) --> SEGMENTATION VIOLATION (SEG1)
...
VA  9: 0x0000001d (decimal:   29) --> SEGMENTATION VIOLATION (SEG0)
```

All 10 addresses are violations, and over 5 seeds × 128 random addresses, **0 of 640** were valid. (Without explicit limits this cannot happen, because the simulator picks each random limit between asize/4 and asize/2.)